WiFi CSI 기반 실내 인원 수 추정 프로젝트 현황 보고서

작성일: 2026 - 07 - 10

프로젝트 경로: `/home/mh/■■■■/project`

1. 프로젝트 목표

본 프로젝트의 최종 목표는 Intel 5300 WiFi NIC에서 수집한 CSI(Channel State Information)를 이용해 실내 공간의 인원 수를 추정하는 것이다.

현재 목표는 사람의 자세나 행동을 세밀하게 추정하는 것이 아니라, 특정 실내 환경에서 `0■`, `1■`, `2■`, `3■`처럼 인원 수 클래스를 분류하는 것이다. 최종 단계에서는 CSI 데이터만 입력으로 사용해 인원 수를 예측하는 모델을 만드는 것을 목표로 한다.

2. 현재까지의 핵심 결론

가장 중요한 성과는 CSI 로그 생성 검증에 성공했다는 점이다.

처음에는 IBSS/ad - hoc 방식과 AP 모드 방식을 시도했지만, 안정적으로 CSI가 생성되지 않았다. 이후 송신 PC와 수신 PC를 같은 휴대폰 핫스팟/AP에 연결하고 송신 PC에서 수신 PC로 UDP 트래픽을 보내는 방식으로 CSI 수집에 성공했다.

현재 성공 조건은 다음과 같이 정리된다.

- 수신 PC는 patched Intel 5300 CSI Tool 드라이버를 사용해야 한다.
- 수신 PC의 `connector_log`가 `1`이어야 한다.
- 송신 PC와 수신 PC가 같은 WiFi AP 또는 휴대폰 핫스팟에 연결되어 있어야 한다.
- 송신 PC에서 수신 PC IP로 충분한 UDP 트래픽을 발생시켜야 한다.
- 수신 PC에서 생성된 `csi.dat` 파일 크기가 0 byte가 아니어야 한다.
- CSI parser로 실제 record count가 확인되어야 한다.

3. 현재 성공한 네트워크/수집 조건

성공한 수집 조건은 다음과 같다.

- AP 방식: 휴대폰 핫스팟
- SSID: `Minhyuck S24+`

- AP BSSID: `92:91:d6:68:b5:14`
- 주파수: 2.4 GHz
- 채널: 6
- 중심 주파수: 2437 MHz
- 수신 PC IP: `10.27.219.22/24`
- 수신 인터페이스: `wlp4s0`
- 수신 PC 커널: `4.15.0-213-generic`
- 수신 드라이버: patched `iwlwifi`
- `connector_log`: `1`
- 송신 방식: UDP 트래픽 송신
- 송신 명령:

```
python3 scripts/sender/safe_sender.py --target 10.27.219.22 --mode udp --duration 60 --rate
100 --size 512
```

4. 실패했던 방식과 원인 정리

4.1 IBSS/ad-hoc 방식

IBSS 방식에서는 송신 PC와 수신 PC가 같은 SSID와 BSSID에 join하는 것까지는 성공했다. `ping`이 성공한 순간도 있었다. 하지만 CSI 데이터가 안정적으로 생성되지 않았다.

초기에는 `fixed-freq`와 고정 BSSID를 함께 사용했을 때 서로 같은 IBSS에 붙은 것처럼 보였지만 실제 station이 잡히지 않거나 ARP가 실패했다. 이후 BSSID를 송신 PC에서 생성한 값으로 맞추고 legacy 방식으로 재시도했을 때 ping은 성공했지만, CSI 생성까지 안정적으로 이어지지는 않았다.

4.2 송신 PC AP 모드 방식

송신 PC의 Intel 5300 인터페이스를 AP로 세우는 방식도 시도했다. 그러나 다음과 같은 오류로 실패했다.

```
command failed: Operation not supported (-95)
nl80211: Failed to set interface to AP mode
```

즉, 현재 장비/드라이버 조합에서는 Intel 5300을 AP 모드로 사용하는 방식이 지원되지 않는 것으로 판단했다.

4.3 USB 테더링만 사용하는 방식

각 PC가 각자의 휴대폰 USB 테더링으로 인터넷에 연결되어 있는 상태에서는 Intel 5300 무선 링크를 통한 CSI 수집이 어렵다. CSI는 Intel 5300 무선 인터페이스로 수신되는 WiFi 프레임에서 생성되기 때문에, USB 테더링 경로로만 데이터가 오가면 CSI가 생기지 않는다.

5. 현재 확보된 CSI 데이터

현재 `data/raw` 아래에는 성공한 CSI 파일 2개와 실패한 0 byte 파일들이 함께 존재한다.

5.1 실패 데이터

다음 세션들은 `csi.dat`가 0 byte이므로 실패 이력으로만 보관한다.

- `session_20260615_203420`
- `session_20260615_203722`
- `session_20260615_205335`
- `session_20260703_151554`
- `session_20260703_151951`
- `session_20260703_172027`

5.2 성공 데이터

현재 성공한 CSI 세션은 다음 2개다.

세션	파일 크기	CSI record 수	라벨	용도
`session_20260710_145820`	약 1.8 MB	4690	2명	검증용
`session_20260710_153743`	약 1.9 MB	4916	2명	검증용

두 세션 모두 실제 CSI가 생성되었고 parser로 record count가 확인되었다. 다만 이 데이터는 3D 프린터 안테나 거치대 완성 전에 수집된 데이터이므로 최종 학습 데이터로는 사용하지 않는 것이 맞다.

현재 두 성공 세션의 `meta.json`에는 다음 기준이 반영되어 있다.

- `quality_status`: `valid_csi`
- `dataset_role`: `verification_only`
- `use_for_final_training`: `false`
- `num_people`: `2`
- `antenna_holder`: `not_used_pre_holder`

6. 현재 구현된 코드

현재 구현된 코드는 크게 6개 영역으로 나뉜다.

6.1 송신 트래픽 생성

파일:

- `scripts/sender/safe_sender.py`

역할:

- 송신 PC에서 수신 PC로 UDP, TCP, ping, iperf3 트래픽을 발생시킨다.
- 현재 CSI 검증 성공에는 UDP 모드를 사용했다.

대표 명령:

```
python3 scripts/sender/safe_sender.py --target 10.27.219.22 --mode udp --duration 60 --rate
100 --size 512
```

6.2 CSI 파싱

파일:

- `scripts/csi/csi_parser.py`

역할:

- Intel 5300 CSI Tool의 `csi.dat` 파일을 Python 표준 라이브러리만으로 파싱한다.
- CSI record 개수, antenna dimension, rate, RSSI, amplitude 통계를 확인한다.
- 실패 파일과 성공 파일을 구분하는 근거를 제공한다.

대표 명령:

```
python3 scripts/csi/csi_parser.py data/raw/session_20260710_153743/csi.dat --pretty
```

6.3 CSI 시각화

파일:

- `scripts/csi/visualize_csi.py`

역할:

- CSI amplitude의 시간 변화, subcarrier heatmap, antenna pair별 평균 amplitude를 SVG/HTML로 생성한다.
- 현재 성공 세션 2개에 대해 시각화 결과가 생성되어 있다.

출력 예:

- `outputs/csi_visualizations/session_20260710_145820/index.html`
- `outputs/csi_visualizations/session_20260710_153743/index.html`

6.4 메타데이터 생성

파일:

- `templates/session_meta_template.json`
- `scripts/dataset/create_meta.py`
- `scripts/dataset/validate_sessions.py`

역할:

- 각 수집 세션마다 `meta.json`을 생성한다.
- 수집 시간, 인원 수 라벨, 네트워크 조건, 송수신 위치, 파일 크기, CSI record count, 최종 학습 사용 여부를 기록한다.
- 전체 세션 목록을 검증하고 `outputs/dataset/session_index.csv`, `outputs/dataset/session_index.json`을 생성한다.

대표 명령:

```
python3 scripts/dataset/validate_sessions.py
```

6.5 전처리

파일:

- `scripts/preprocess/make_windows.py`
- `scripts/preprocess/extract_features.py`

역할:

- `make_windows.py`는 세션 단위 CSI를 일정 시간 윈도우로 나눈다.
- 기본 설정은 2초 window, 1초 stride다.
- `extract_features.py`는 각 윈도우에서 amplitude, RSSI, noise, AGC, antenna pair, subcarrier 기반 통계 feature를 추출한다.

현재 검증용 dry - run 결과:

- `data/processed/windows_verification.csv`: 118개 window
- `data/processed/features_verification.csv`: 118개 feature row

6.6 baseline 학습/평가

파일:

- `scripts/train/train_baseline.py`
- `scripts/evaluate/evaluate_model.py`

역할:

- `train_baseline.py`는 nearest centroid 또는 majority baseline 모델을 학습한다.
- `evaluate_model.py`는 저장된 모델과 feature CSV를 이용해 평가한다.

현재 검증용 dry - run 결과:

- `outputs/models/baseline_verification_model.json`
- `outputs/models/baseline_verification_split.json`
- `outputs/evaluation/baseline_verification_metrics.json`
- `outputs/evaluation/model_verification_metrics.json`

현재 dry - run에서는 accuracy와 macro F1이 1.0으로 나오지만, 이 결과는 성능 지표로 해석하면 안 된다. 현재 검증 데이터는 모두 `2■` 라벨 하나뿐이므로 모델이 실제로 인원 수를 구분했다고 볼 수 없다. 이 결과는 코드 파이프라인이 끝까지 실행된다는 검증 의미만 가진다.

7. 현재 작성된 문서

현재 작성된 주요 문서는 다음과 같다.

- `docs/csi_capture_success_conditions_20260710.md`
- `docs/csi_capture_success_conditions_20260710.pdf`
- `docs/data_success_failure_criteria.md`
- `docs/data_success_failure_criteria.pdf`
- `docs/meta_json_schema.md`
- `docs/csi_parsing_visualization.md`
- `docs/training_pipeline_skeleton.md`
- `docs/midterm_progress_report.md`

이 중 성공 조건 문서와 성공/실패 데이터 구분 문서는 PDF로도 변환되어 있다.

8. 본 데이터 수집 전 준비해야 할 것

현재는 CSI 수집 가능 여부와 코드 파이프라인 뼈대 구현이 완료된 상태다. 하지만 최종 학습에 사용할 데이터 수집은 아직 시작하지 않은 상태로 보는 것이 맞다.

본 데이터 수집 전에 반드시 정해야 할 항목은 다음과 같다.

8.1 실험 프로토콜

아직 최종 실험 프로토콜은 확정되지 않았다. 팀원들과 상의해서 다음 항목을 정해야 한다.

- 수집할 인원 수 클래스: 예를 들어 `0명`, `1명`, `2명`, `3명`
- 각 클래스별 반복 횟수
- 각 세션 길이: 예를 들어 60초, 90초, 120초 등
- 사람의 위치 조건: 고정 위치, 자유 위치, 랜덤 위치 중 선택
- 사람의 활동 조건: 정지, 자연스러운 움직임, 혼합 조건 중 선택
- 문 개폐 여부
- 주변 사람 출입 통제 기준
- 수집 순서 랜덤화 여부
- train/test 분리 기준
- 같은 날/다른 날 수집 데이터를 어떻게 나눌지

현재 코드의 기본 전처리 설정은 2초 window, 1초 stride이며, 60초 세션 1개에서 약 59개 window가 생성된다. 따라서 각 클래스별로 충분한 반복 세션을 수집해야 한다.

8.2 안테나 거치대

3D 프린터로 제작할 안테나 거치대가 아직 최종 실험에 반영되지 않았다. 현재 거치대 스펙은 다음 파일에 정리되어 있다.

- `antenna_3D_Printer/README.md`
- `antenna_3D_Printer/antenna_mount.scad`

현재 설계의 핵심 기준은 다음과 같다.

- 안테나 3개 사용
- 안테나 간격 62.5 mm
- 2.4 GHz 반파장 기준
- TX와 RX 모두 동일한 거치대 구조 사용
- 안테나 방향과 높이를 고정해 CSI 데이터의 재현성을 높이는 것이 목적

중요한 점은 거치대가 완성된 뒤의 데이터만 최종 학습 데이터로 사용해야 한다는 것이다. 거치대 제작 전 데이터는 안테나 위치와 각도가 달라 최종 모델 학습에 섞으면 데이터 분포가 달라질 수 있다.

8.3 실험 공간 배치

직사각형 방을 기준으로 할 때, 송신 PC와 수신 PC는 가능한 한 서로 마주 보는 양쪽 벽 근처에 배치하는 것이 좋다. 특히 짧은 변 방향 양 끝에 놓으면 링크 거리가 짧아지고 신호 안정성이 좋아질 가능성이 있다.

다만 최종 배치는 실험 프로토콜에서 고정해야 한다. 한 번 정한 위치, 높이, 방향은 데이터 수집 중 바꾸지 않는 것이 중요하다.

9. 나중에 실제 데이터를 수집하는 절차

거치대와 실험 프로토콜이 확정되면 다음 순서로 본 데이터를 수집하면 된다.

9.1 수집 전 확인

수신 PC에서 확인:

```
iw dev wlp4s0 link
ip -4 addr show wlp4s0
cat /sys/module/iwlwifi/parameters/connector_log
```

송신 PC에서 확인:

```
iw dev wlp4s0 link
ip -4 addr show wlp4s0
ping -c 4 <■■_PC_IP>
```

성공 조건:

- 두 PC가 같은 AP/핫스팟에 연결되어 있어야 한다.
- 송신 PC에서 수신 PC로 ping이 성공해야 한다.
- 수신 PC에서 `connector_log`가 `1`이어야 한다.

9.2 수신 PC에서 CSI 로깅 시작

수신 PC에서 CSI 로깅 스크립트를 실행한다.

```
bash tools/start_csi_logging.sh
```

새 세션 폴더가 생성되고 `data/raw/session_*/csi.dat` 파일이 만들어지는지 확인한다.

9.3 송신 PC에서 트래픽 발생

송신 PC에서 수신 PC IP를 대상으로 UDP 트래픽을 보낸다.

```
python3 scripts/sender/safe_sender.py --target <■■_PC_IP> --mode udp --duration 60 --rate
100 --size 512
```

9.4 수집 완료 후 파일 확인

수신 PC에서 확인:

```
ls -lh data/raw/session_*/csi.dat
python3 scripts/csi/csi_parser.py <■■■■>/csi.dat --pretty
```

성공 기준:

- `csi.dat`가 0 byte가 아니어야 한다.
- parser에서 record count가 0보다 커야 한다.

9.5 메타데이터 작성

각 세션마다 `meta.json`을 작성해야 한다.

검증용 데이터가 아니라 최종 학습용 데이터라면 다음 값이 들어가야 한다.

```
{
  "quality_status": "valid_csi",
  "dataset_role": "training_candidate",
  "use_for_final_training": true,
  "label": {
    "num_people": 0
  }
}
```

`num_people`는 해당 세션의 실제 인원 수로 바꿔야 한다.

10. 실제 전처리 및 학습 절차

본 데이터 수집이 끝나면 다음 순서로 처리한다.

10.1 전체 세션 검증

```
python3 scripts/dataset/validate_sessions.py
```

출력:

- `outputs/dataset/session_index.csv`
- `outputs/dataset/session_index.json`

10.2 윈도우 생성

최종 학습 후보 데이터만 사용:

```
python3 scripts/preprocess/make_windows.py
```

출력:

- `data/processed/windows.csv`
- `data/processed/windows.json`

검증용 데이터까지 포함해서 테스트할 때:

```
python3 scripts/preprocess/make_windows.py --include-verification --out-csv data/processed/
windows_verification.csv --out-json data/processed/windows_verification.json
```

10.3 feature 추출

```
python3 scripts/preprocess/extract_features.py --windows-csv data/processed/windows.csv --
out-csv data/processed/features.csv
```

10.4 baseline 모델 학습

```
python3 scripts/train/train_baseline.py --features-csv data/processed/features.csv
```

출력:

- `outputs/models/baseline_model.json`
- `outputs/models/baseline_split.json`
- `outputs/evaluation/baseline_predictions.csv`
- `outputs/evaluation/baseline_metrics.json`

10.5 모델 평가

```
python3 scripts/evaluate/evaluate_model.py --features-csv data/processed/features.csv --
model outputs/models/baseline_model.json --split outputs/models/baseline_split.json --mode
test
```

11. 현재 상태 요약

현재 프로젝트 상태는 다음과 같이 정리할 수 있다.

- Intel 5300 CSI 로그 생성 성공
- 휴대폰 핫스팟 기반 AP 방식으로 CSI 수집 성공
- 성공 세션 2개 확보
- 실패 세션과 성공 세션 구분 기준 정리 완료

- `meta.json` 양식 구현 완료
- CSI parser 구현 완료
- CSI 시각화 코드 구현 완료
- 세션 검증 코드 구현 완료
- 윈도우 생성 코드 구현 완료
- feature 추출 코드 구현 완료
- baseline 학습/평가 코드 뼈대 구현 완료
- 현재 학습 결과는 검증용 dry - run이며 실제 성능으로 해석하면 안 됨
- 최종 실험 프로토콜은 아직 확정 필요
- 3D 프린터 안테나 거치대 제작 후 본 데이터 수집 필요

12. 앞으로 해야 할 일

다음 단계는 코드 구현보다 실험 설계와 데이터 수집의 품질을 고정하는 것이 중요하다.

우선순위는 다음과 같다.

1. 3D 프린터 안테나 거치대 제작 및 TX/RX 장착
2. 송신/수신 위치, 높이, 안테나 방향 고정
3. 실험 프로토콜 확정
4. `0■`, `1■`, `2■`, `3■` 등 목표 클래스별 본 데이터 수집
5. 각 세션별 `meta.json` 작성
6. 세션 검증
7. 전처리 및 feature 추출
8. baseline 모델 학습
9. train/test 성능 확인
10. 필요 시 모델 고도화

현재 프로젝트는 “ CSI 수집 가능성 확인 ” 과 “ 데이터가 들어왔을 때 처리할 코드 뼈대 ” 까지 완료된 상태다. 다음 병목은 코드가 아니라, 재현 가능한 실험 프로토콜과 충분한 양의 본 데이터 수집이다.